

循序渐进，学习开发 xv6-riscv

第 10 章 抢占式多任务

汪辰



目录

01

抢占式多任务（复习）

02

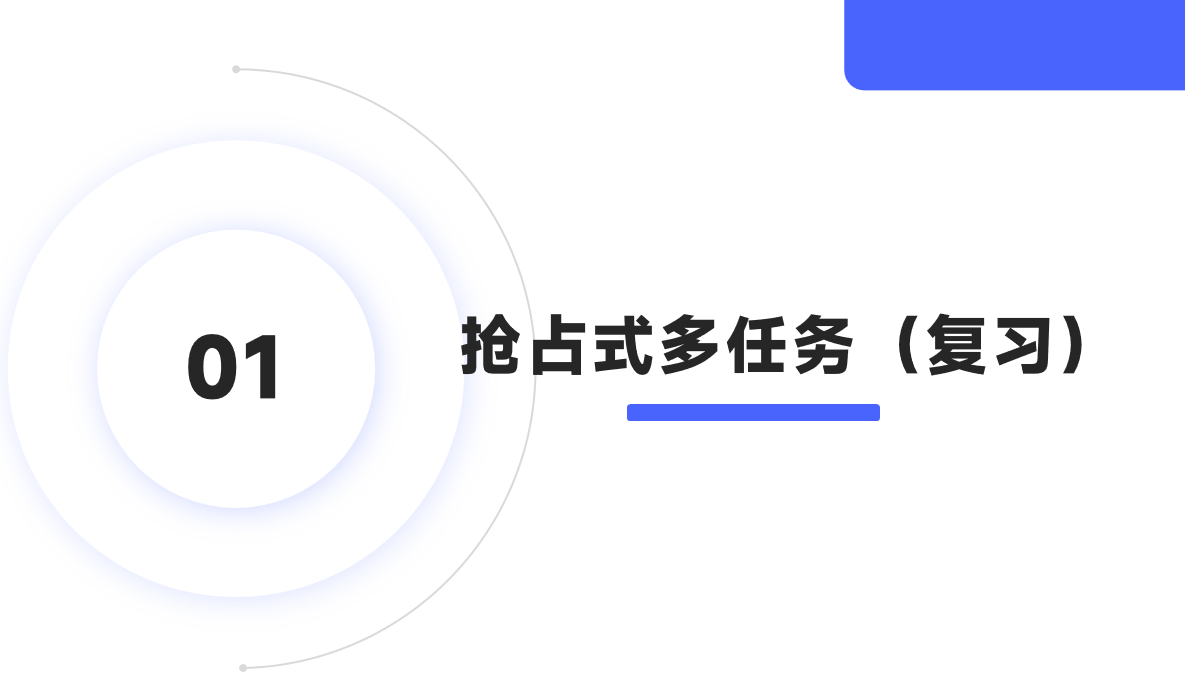
抢占式多任务设计

03

改进用户任务的启动方式

04

抢占式多任务运行剖析



01

抢占式多任务（复习）

抢占式多任务（复习）

- **协作式多任务 (Cooperative Multi-Tasking)**: 正在处理器上运行的用户任务主动放弃处理器，将处理器的控制权交还给操作系统，这样操作系统就可以执行相关的代码并选择和运行其他的用户任务。也就是说，多个任务相互协作，交替运行和放弃处理器，达到整体系统上多个任务并发运行的效果。
- **抢占式多任务 (Preemptive Multi-Tasking)**: 和协作方式相反，操作系统并不信任用户任务会合理地主动放弃处理器，所以内核通过其他手段强制剥夺正在处理器上运行的用户任务对处理器的使用权，将处理器分配给其他的任务实现交替运行和放弃处理器，同样达到整体系统上多个任务并发运行的效果。

A decorative graphic on the left side of the slide. It features two concentric circles with a light blue glow. A thin grey arc with dots at its ends is positioned around the circles. In the top right corner, there is a solid blue rectangle.

02

抢占式多任务设计

抢占式多任务设计

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    intr_on();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

```
void user_task1(void)
{
    printf("Task 1: Created!\n");
    while (1) {
        printf("Task 1: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```

xv6 kernel is booting

```
Task 0: Created!
Task 0: Running...
Task 1: Created!
Task 1: Running...
Task 0: Running...
Task 1: Running...
Task 0: Running...
Task 1: Running...
Task 0: Running...
Task 1: Running...
```

抢占式多任务设计

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
    }
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    intr_on();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

```
void user_task1(void)
{
    printf("Task 1: Created!\n");
    while (1) {
        printf("Task 1: Running...\n");
        task_delay(DELAY);
    }
}
```

xv6 kernel is booting

```
Task 0: Created!
Task 0: Running...
Task 1: Created!
Task 1: Running...
Task 0: Running...
Task 1: Running...
Task 0: Running...
Task 1: Running...
Task 0: Running...
Task 1: Running...
```

抢占式多任务设计

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
    }
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    intr_on();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

```
void user_task1(void)
{
    printf("Task 1: Created!\n");
    while (1) {
        printf("Task 1: Running...\n");
        task_delay(DELAY);
    }
}
```

```
kernelvec:
    # save context .....
    call kerneltrap
```

```
void kerneltrap()
{
    int which_dev = 0;
    // .....
    which_dev = devintr();
    // give up the CPU if a timer interrupt.
    if(which_dev == 2 && myproc() != 0)
        yield();
    // .....
}
```

xv6 kernel is booting

```
Task 0: Created!
Task 0: Running...
Task 1: Created!
Task 1: Running...
Task 0: Running...
Task 1: Running...
Task 0: Running...
Task 1: Running...
Task 0: Running...
Task 1: Running...
```

抢占式多任务设计

```
// interrupts and exceptions from kernel code go here via kernelvec,  
// on whatever the current kernel stack is.  
void kerneltrap()  
{  
    int which_dev = 0;  
    uint64 sepc = r_sepc();  
    uint64 sstatus = r_sstatus();  
    // .....  
    which_dev = devintr();  
    // .....  
    // give up the CPU if this is a timer interrupt.  
    if(which_dev == 2 && myproc() != 0)  
        yield();  
    // the yield() may have caused some traps to occur,  
    // so restore trap registers for use by kernelvec.S's sepc instruction.  
    w_sepc(sepc);  
    w_sstatus(sstatus);  
}
```

抢占式多任务设计

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
    }
}
```



```
kernelvec:
    # save context .....
    call kerneltran
    # restore context .....
    sret
```

```
void kerneltrap()
{
    int which_dev = 0;
    // .....
    which_dev = devintr();
    // give up the CPU if a timer interrupt.
    if(which_dev == 2 && myproc() != 0)
        yield();
    // .....
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    intr_on();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            swtch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

```
void user_task1(void)
{
    printf("Task 1: Created!\n");
    while (1) {
        printf("Task 1: Running...\n");
        task_delay(DELAY);
    }
}
```

xv6 kernel is booting

```
Task 0: Created!
Task 0: Running...
Task 1: Created!
Task 1: Running...
Task 0: Running...
Task 1: Running...
Task 0: Running...
Task 1: Running...
Task 0: Running...
Task 1: Running...
```

抢占式多任务设计

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
    }
}
```



```
kernelvec:
    # save context .....
    call kerneltran
    # restore context .....
    sret
```

```
void kerneltrap()
{
    int which_dev = 0;
    // .....
    which_dev = devintr();
    // give up the CPU if a timer interrupt.
    if(which_dev == 2 && myproc() != 0)
        yield();
    // .....
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c;
    intr_on();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```



```
void user_task1(void)
{
    printf("Task 1: Created!\n");
    while (1) {
        printf("Task 1: Running...\n");
        task_delay(DELAY);
    }
}
```

xv6 kernel is booting

```
Task 0: Created!
Task 0: Running...
Task 1: Created!
Task 1: Running...
Task 0: Running...
Task 1: Running...
Task 0: Running...
Task 1: Running...
Task 0: Running...
Task 1: Running...
```



03

改进用户任务的启动方式

改进用户任务的启动方式

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    intr_on();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            swtch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

改进用户任务的启动方式

```
static struct proc*  
allocproc(void (*start_routine)(void))  
{  
    // .....  
    p->state = USED;  
    // Set up new context to start executing  
    memset(&p->context, 0, sizeof(p->context));  
    p->context.ra = (uint64)start_routine;  
    p->context.sp = p->kstack + PGSIZE;  
    return p;  
}
```

```
void scheduler(void)  
{  
    struct proc *p;  
    struct cpu *c = mycpu();  
    intr_on();  
    c->proc = 0;  
    for(;;){  
        for(p = proc; p < &proc[NPROC]; p++) {  
            // .....  
            c->proc = p;  
            swtch(&c->context, &p->context);  
            c->proc = 0;  
            // .....  
        }  
    }  
}
```

```
# void swtch(struct context *old,  
#           struct context *new);  
.globl swtch  
swtch:  
    sd ra, 0(a0)  
    // save .....  
    sd s11, 104(a0)  
    ld ra, 0(a1)  
    // restore .....  
    ld s11, 104(a1)  
    ret
```

jalr x0, 0(ra)

改进用户任务的启动方式

```
static struct proc*
allocproc(void (*start_routine)(void))
{
    // .....
    p->state = USED;
    // Set up new context to start executing
    memset(&p->context, 0, sizeof(p->context));
    p->context.ra = (uint64)start_routine;
    p->context.sp = p->kstack + PGSIZE;
    return p;
}
```

改进用户任务的启动方式

```
static struct proc*
allocproc(void (*start_routine)(void))
{
    // .....
    p->state = USED;
    // Set up new context to start executing
    memset(&p->context, 0, sizeof(p->context));
    p->context.ra = (uint64)start_routine;
    p->context.sp = p->kstack + PGSIZE;
    return p;
}
```



```
static struct proc*
allocproc(void (*start_routine)(void))
{
    // .....
    p->state = USED;
    // Set up new context to start executing
    memset(&p->context, 0, sizeof(p->context));
    p->context.ra = (uint64)kerneltrapret;
    p->context.sp = p->kstack + PGSIZE;
    p->start = start_routine;
    return p;
}
```

改进用户任务的启动方式

```
static struct proc*
allocproc(void (*start_routine)(void))
{
    // .....
    p->state = USED;
    // Set up new context to start executing
    memset(&p->context, 0, sizeof(p->context));
    p->context.ra = (uint64)start_routine;
    p->context.sp = p->kstack + PGSIZE;
    return p;
}
```



```
static struct proc*
allocproc(void (*start_routine)(void))
{
    // .....
    p->state = USED;
    // Set up new context to start executing
    memset(&p->context, 0, sizeof(p->context));
    p->context.ra = (uint64)kerneltrapret;
    p->context.sp = p->kstack + PGSIZE;
    p->start = start_routine;
    return p;
}
```

```
void kerneltrapret(void)
{
    struct proc *p = myproc();
    unsigned long x = r_sstatus();
    x |= SSTATUS_SPP;
    x |= SSTATUS_SPIE;
    w_sstatus(x);
    w_sepc((uint64)p->start);
    asm volatile("sret");
}
```

改进用户任务的启动方式

```
static struct proc*
allocproc(void (*start_routine)(void))
{
    // .....
    p->state = USED;
    // Set up new context to start executing
    memset(&p->context, 0, sizeof(p->context));
    p->context.ra = (uint64)start_routine;
    p->context.sp = p->kstack + PGSIZE;
    return p;
}
```



```
static struct proc*
allocproc(void (*start_routine)(void))
{
    // .....
    p->state = USED;
    // Set up new context to start executing
    memset(&p->context, 0, sizeof(p->context));
    p->context.ra = (uint64)kerneltrapret;
    p->context.sp = p->kstack + PGSIZE;
    p->start = start_routine;
    return p;
}
```

```
31 // Per-process state
32 struct proc {
33     enum procstate state;
34     int pid;
35
36     uint64 kstack;
37     struct context context;
38 };
39
```

```
31 // Per-process state
32 struct proc {
33     enum procstate state;
34     int pid;
35
36     uint64 kstack;
37     struct context context:
38+     void (*start)(void);
39 };
40
```

```
void kerneltrapret(void)
{
    struct proc *p = myproc();
    unsigned long x = r_sstatus();
    x |= SSTATUS_SPP;
    x |= SSTATUS_SPIE;
    w_sstatus(x);
    w_sepc((uint64)p->start);
    asm volatile("sret");
}
```

改进用户任务的启动方式

```
static struct proc*
allocproc(void (*start_routine)(void))
{
    // .....
    p->state = USED;
    // Set up new context to start executing
    memset(&p->context, 0, sizeof(p->context));
    p->context.ra = (uint64)start_routine;
    p->context.sp = p->kstack + PGSIZE;
    return p;
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    intr_on();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

```
static struct proc*
allocproc(void (*start_routine)(void))
{
    // .....
    p->context.ra = (uint64)kerneltrapret;
    p->context.sp = p->kstack + PGSIZE;
    return p;
}
```

```
31 // Per-process state
32 struct proc {
33     enum procstate state;
34     int pid;
35
36     uint64 kstack;
37     struct context context;
38 };
39
```

```
32 struct proc {
33     enum procstate state;
34     int pid;
35
36     uint64 kstack;
37     struct context context;
38+    void (*start)(void);
39 };
40
```

```
static void kerneltrapret(void)
{
    struct proc *p = myproc();
    unsigned long x = r_sstatus();
    x |= SSTATUS_SPP;
    x |= SSTATUS_SPIE;
    w_sstatus(x);
    w_sepc((uint64)p->start);
    asm volatile("sret");
}
```

改进用户任务的启动方式

```
static struct proc*
allocproc(void (*start_routine)(void))
{
    // .....
    p->state = USED;
    // Set up new context to start executing
    memset(&p->context, 0, sizeof(p->context));
    p->context.ra = (uint64)start_routine;
    p->context.sp = p->kstack + PGSIZE;
    return p;
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

```
static struct proc*
allocproc(void (*start_routine)(void))
{
    // .....
    // Set up new context to start executing
    memset(&p->context, 0, sizeof(p->context));
    p->context.ra = (uint64)kerneltrapret;
    p->context.sp = p->kstack + PGSIZE;
    return p;
}
```

```
31 // Per-process state
32 struct proc {
33     enum procstate state;
34     int pid;
35
36     uint64 kstack;
37     struct context context;
38 };
39
```

```
32 struct proc {
33     enum procstate state;
34     int pid;
35
36     uint64 kstack;
37     struct context context;
38     void (*start)(void);
39 };
40
```

```
static void kerneltrapret(void)
{
    struct proc *p = myproc();
    unsigned long x = r_sstatus();
    x |= SSTATUS_SPP;
    x |= SSTATUS_SPIE;
    w_sstatus(x);
    w_sepc((uint64)p->start);
    asm volatile("sret");
}
```


改进用户任务的启动方式 - 流程剖析



```
void main()
{
    // .....
    printf("xv6 kernel is booting\n");
    printf("\n");
    // .....
    userinit();
    scheduler();
}
```



改进用户任务的启动方式 - 流程剖析



```
void main()
{
    // .....
    printf("xv6 kernel is booting\n");
    printf("\n");
    // .....
    userinit();
    scheduler();
}
```

xv6 kernel is booting

改进用户任务的启动方式 - 流程剖析

```
void main()
{
    // .....
    printf("xv6 kernel is booting\n");
    printf("\n");
    // .....
    userinit();
    scheduler();
}
```



xv6 kernel is booting

改进用户任务的启动方式 - 流程剖析

```
void main()
```

```
{
```

```
    // .....
```

```
    printf("xv6 kernel is booting\n");
```

```
    printf("\n");
```

```
    // .....
```

```
    userinit();
```

```
    scheduler();
```

```
}
```

```
void userinit(void)
```

```
{
```

```
    struct proc *p;
```

```
    p = allocproc(user_task0);
```

```
    p->state = RUNNABLE;
```

```
    p = allocproc(user_task1);
```

```
    p->state = RUNNABLE;
```

```
}
```

xv6 kernel

改进用户任务的启动方式 - 流程剖析



```
static struct proc*
allocproc(void (*start_routine)(void))
{
    // .....
    p->state = USED;
    // Set up new context to start executing
    memset(&p->context, 0, sizeof(p->context));
    p->context.ra = (uint64)kerneltrapret;
    p->context.sp = p->kstack + PGSIZE;
    p->start = start_routine;
    return p;
}
```

```
void main()
{
    // .....
    printf("xv6 kernel is booting\n");
    printf("\n");
    // .....
    userinit();
    scheduler();
}
```

```
void userinit(void)
{
    struct proc *p;
    p = allocproc(user_task0);
    p->state = RUNNABLE;
    p = allocproc(user_task1);
    p->state = RUNNABLE;
}
```

xv6 kernel

改进用户任务的启动方式 - 流程剖析

```
static struct proc*
allocproc(void (*start_routine)(void))
{
    // .....
    p->state = USED;
    // Set up new context to start executing
    memset(&p->context, 0, sizeof(p->context));
    p->context.ra = (uint64)kerneltrapret;
    p->context.sp = p->kstack + PGSIZE;
    p->start = start_routine;
    return p;
}
```

```
void kerneltrapret(void)
{
    struct proc *p = myproc();
    unsigned long x = r_sstatus();
    x |= SSTATUS_SPP;
    x |= SSTATUS_SPIE;
    w_sstatus(x);
    w_sepc((uint64)p->start);
    asm volatile("sret");
}
```

```
void main()
{
    // .....
    printf("xv6 kernel is booting\n");
    printf("\n");
    // .....
    userinit();
    scheduler();
}
```

```
void userinit(void)
{
    struct proc *p;
    p = allocproc(user_task0);
    p->state = RUNNABLE;
    p = allocproc(user_task1);
    p->state = RUNNABLE;
}
```

xv6 kernel

改进用户任务的启动方式 - 流程剖析

```
static struct proc*
allocproc(void (*start_routine)(void))
{
    // .....
    p->state = USED;
    // Set up new context to start executing
    memset(&p->context, 0, sizeof(p->context));
    p->context.ra = (uint64)kerneltrapret;
    p->context.sp = p->kstack + PGSIZE;
    p->start = start_routine;
    return p;
}
```

```
void kerneltrapret(void)
{
    struct proc *p = myproc();
    unsigned long x = r_sstatus();
    x |= SSTATUS_SPP;
    x |= SSTATUS_SPIE;
    w_sstatus(x);
    w_sepc((uint64)p->start);
    asm volatile("sret");
}
```

```
void main()
{
    // .....
    printf("xv6 kernel is booting\n");
    printf("\n");
    // .....
    userinit();
    scheduler();
}
```

```
void userinit(void)
{
    struct proc *p;
    p = allocproc(user_task0);
    p->state = RUNNABLE;
    p = allocproc(user_task1);
    p->state = RUNNABLE;
}
```

xv6 kernel

改进用户任务的启动方式 - 流程剖析

```
static struct proc*
allocproc(void (*start_routine)(void))
{
    // .....
    p->state = USED;
    // Set up new context to start executing
    memset(&p->context, 0, sizeof(p->context));
    p->context.ra = (uint64)kerneltrapret;
    p->context.sp = p->kstack + PGSIZE;
    p->start = start_routine;
    return p;
}
```

```
void kerneltrapret(void)
{
    struct proc *p = myproc();
    unsigned long x = r_sstatus();
    x |= SSTATUS_SPP;
    x |= SSTATUS_SPIE;
    w_sstatus(x);
    w_sepc((uint64)p->start);
    asm volatile("sret");
}
```

```
void main()
{
    // .....
    printf("xv6 kernel is booting\n");
    printf("\n");
    // .....
    userinit();
    scheduler();
}
```

```
void userinit(void)
{
    struct proc *p;
    p = allocproc(user_task0);
    p->state = RUNNABLE;
    p = allocproc(user_task1);
    p->state = RUNNABLE;
}
```

xv6 kernel

改进用户任务的启动方式 - 流程剖析

```
static struct proc*
allocproc(void (*start_routine)(void))
{
    // .....
    p->state = USED;
    // Set up new context to start executing
    memset(&p->context, 0, sizeof(p->context));
    p->context.ra = (uint64)kerneltrapret;
    p->context.sp = p->kstack + PGSIZE;
    p->start = start_routine;
    return p;
}
```

```
void main()
{
    // .....
    printf("xv6 kernel is booting\n");
    printf("\n");
    // .....
    userinit();
    scheduler();
}
```

```
void kerneltrapret(void)
{
    struct proc *p = myproc();
    unsigned long x = r_sstatus();
    x |= SSTATUS_SPP;
    x |= SSTATUS_SPIE;
    w_sstatus(x);
    w_sepc((uint64)p->start);
    asm volatile("sret");
}
```

xv6 kernel is booting

改进用户任务的启动方式 - 流程剖析

```
static struct proc*
allocproc(void (*start_routine)(void))
{
    // .....
    p->state = USED;
    // Set up new context to start executing
    memset(&p->context, 0, sizeof(p->context));
    p->context.ra = (uint64)kerneltrapret;
    p->context.sp = p->kstack + PGSIZE;
    p->start = start_routine;
    return p;
}
```

```
void kerneltrapret(void)
{
    struct proc *p = myproc();
    unsigned long x = r_sstatus();
    x |= SSTATUS_SPP;
    x |= SSTATUS_SPIE;
    w_sstatus(x);
    w_sepc((uint64)p->start);
    asm volatile("sret");
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            swtch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

xv6 kernel is booting

改进用户任务的启动方式 - 流程剖析

```
static struct proc*
allocproc(void (*start_routine)(void))
{
    // .....
    p->state = USED;
    // Set up new context to start executing
    memset(&p->context, 0, sizeof(p->context));
    p->context.ra = (uint64)kerneltrapret;
    p->context.sp = p->kstack + PGSIZE;
    p->start = start_routine;
    return p;
}
```

```
void kerneltrapret(void)
{
    struct proc *p = myproc();
    unsigned long x = r_sstatus();
    x |= SSTATUS_SPP;
    x |= SSTATUS_SPIE;
    w_sstatus(x);
    w_sepc((uint64)p->start);
    asm volatile("sret");
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            swtch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

xv6 kernel is booting

改进用户任务的启动方式 - 流程剖析

```
static struct proc*
allocproc(void (*start_routine)(void))
{
    // .....
    p->state = USED;
    // Set up new context to start executing
    memset(&p->context, 0, sizeof(p->context));
    p->context.ra = (uint64)kerneltrapret;
    p->context.sp = p->kstack + PGSIZE;
    p->start = start_routine;
    return p;
}
```

```
void kerneltrapret(void)
{
    struct proc *p = myproc();
    unsigned long x = r_sstatus();
    x |= SSTATUS_SPP;
    x |= SSTATUS_SPIE;
    w_sstatus(x);
    w_sepc((uint64)p->start);
    asm volatile("sret");
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            swtch(&c->context, &p->context);
            // .....
        }
    }
}
```

jalr x0, 0(ra)

xv6 kernel is booting

改进用户任务的启动方式 - 流程剖析

```
static struct proc*
allocproc(void (*start_routine)(void))
{
    // .....
    p->state = USED;
    // Set up new context to start executing
    memset(&p->context, 0, sizeof(p->context));
    p->context.ra = (uint64)kerneltrapret;
    p->context.sp = p->kstack + PGSIZE;
    p->start = start_routine;
    return p;
}
```

```
void kerneltrapret(void)
{
    struct proc *p = myproc();
    unsigned long x = r_sstatus();
    x |= SSTATUS_SPP;
    x |= SSTATUS_SPIE;
    w_sstatus(x);
    w_sepc((uint64)p->start);
    asm volatile("sret");
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            swtch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

xv6 kernel is booting

改进用户任务的启动方式 - 流程剖析

```
static struct proc*
allocproc(void (*start_routine)(void))
{
    // .....
    p->state = USED;
    // Set up new context to start executing
    memset(&p->context, 0, sizeof(p->context));
    p->context.ra = (uint64)kerneltrapret;
    p->context.sp = p->kstack + PGSIZE;
    p->start = start_routine;
    return p;
}
```

```
void kerneltrapret(void)
{
    struct proc *p = myproc();
    unsigned long x = r_sstatus();
    x |= SSTATUS_SPP;
    x |= SSTATUS_SPIE;
    w_sstatus(x);
    w_sepc((uint64)p->start);
    asm volatile("sret");
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            swtch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

xv6 kernel is booting

改进用户任务的启动方式 - 流程剖析



```
void user_task0(void)
```

```
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
    }
}
```

```
void kerneltrapret(void)
```

```
{
    struct proc *p = myproc();
    unsigned long x = r_sstatus();
    x |= SSTATUS_SPP;
    x |= SSTATUS_SPIE;
    w_sstatus(x);
    w_sepc((uint64)p->start);
    asm volatile("sret");
}
```

```
void scheduler(void)
```

```
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```



xv6 kernel is booting

改进用户任务的启动方式 - 流程剖析

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
    }
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

xv6 kernel is booting

Task 0: Created!

改进用户任务的启动方式 - 流程剖析

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
    }
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

xv6 kernel is booting

```
Task 0: Created!
Task 0: Running...
```

改进用户任务的启动方式 - 流程剖析

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
    }
}
```

```
kernelvec:
    # save context .....
    call kerneltrap
    # restore context .....
    sret
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

xv6 kernel is booting

```
Task 0: Created!
Task 0: Running...
```

改进用户任务的启动方式 - 流程剖析

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
    }
}
```

```
kernelvec:
    # save context .....
    call kerneltrap
    # restore context .....
    sret
```

```
void kerneltrap()
{
    int which_dev = 0;
    // .....
    which_dev = devintr();
    // give up the CPU if a timer interrupt.
    if(which_dev == 2 && myproc() != 0)
        yield();
    // .....
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

xv6 kernel is booting

Task 0: Created!
Task 0: Running...

改进用户任务的启动方式 - 流程剖析

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
    }
}
```

```
kernelvec:
    # save context .....
    call kerneltrap
    # restore context .....
    sret
```

```
void kerneltrap()
{
    int which_dev = 0;
    // .....
    which_dev = devintr();
    // give up the CPU if a timer interrupt.
    if(which_dev == 2 && myproc() != 0)
        yield();
    // .....
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            swtch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

xv6 kernel is booting

Task 0: Created!
Task 0: Running...

改进用户任务的启动方式 - 流程剖析

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
    }
}
```

```
kernelvec:
    # save context .....
    call kerneltrap
    # restore context .....
    sret
```

```
void kerneltrap()
{
    int which_dev = 0;
    // .....
    which_dev = devintr();
    // give up the CPU if a timer interrupt.
    if(which_dev == 2 && myproc() != 0)
        yield();
    // .....
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            swtch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

xv6 kernel is booting

Task 0: Created!
Task 0: Running...

改进用户任务的启动方式 - 流程剖析

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
    }
}
```

```
kernelvec:
    # save context .....
    call kerneltrap
    # restore context .....
    sret
```

```
void kerneltrap()
{
    int which_dev = 0;
    // .....
    which_dev = devintr();
    // give up the CPU if a timer interrupt.
    if(which_dev == 2 && myproc() != 0)
        yield();
    // .....
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            swtch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

xv6 kernel is booting

Task 0: Created!
Task 0: Running...

改进用户任务的启动方式 - 流程剖析

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
    }
}
```

```
kernelvec:
    # save context .....
    call kerneltrap
    # restore context .....
    sret
```

```
void kerneltrap()
{
    int which_dev = 0;
    // .....
    which_dev = devintr();
    // give up the CPU if a timer interrupt.
    if(which_dev == 2 && myproc() != 0)
        yield();
    // .....
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

xv6 kernel is booting

Task 0: Created!
Task 0: Running...

```
static struct proc*
allocproc(void (*start_routine)(void))
{
    // .....
    p->state = USED;
    // Set up new context to start executing
    memset(&p->context, 0, sizeof(p->context));
    p->context.ra = (uint64)kerneltrapret;
    p->context.sp = p->kstack + PGSIZE;
    p->start = start_routine;
    return p;
}
```

```
void kerneltrapret(void)
{
    struct proc *p = myproc();
    unsigned long x = r_sstatus();
    x |= SSTATUS_SPP;
    x |= SSTATUS_SPIE;
    w_sstatus(x);
    w_sepc((uint64)p->start);
    asm volatile("sret");
}
```

改进用户任务的启动方式 - 流程剖析

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
    }
}
```

```
kernelvec:
    # save context .....
    call kerneltrap
    # restore context .....
    sret
```

```
void kerneltrap()
{
    int which_dev = 0;
    // .....
    which_dev = devintr();
    // give up the CPU if a timer interrupt.
    if(which_dev == 2 && myproc() != 0)
        yield();
    // .....
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

xv6 kernel is booting

Task 0: Created!
Task 0: Running...

```
static struct proc*
allocproc(void (*start_routine)(void))
{
    // .....
    p->state = USED;
    // Set up new context to start executing
    memset(&p->context, 0, sizeof(p->context));
    p->context.ra = (uint64)kerneltrapret;
    p->context.sp = p->kstack + PGSIZE;
    p->start = start_routine;
    return p;
}
```

```
void kerneltrapret(void)
{
    struct proc *p = myproc();
    unsigned long x = r_sstatus();
    x |= SSTATUS_SPP;
    x |= SSTATUS_SPIE;
    w_sstatus(x);
    w_sepc((uint64)p->start);
    asm volatile("sret");
}
```


改进用户任务的启动方式 - 流程剖析

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
    }
}
```

```
kernelvec:
    # save context .....
    call kerneltrap
    # restore context .....
    sret
```

```
void kerneltrap()
{
    int which_dev = 0;
    // .....
    which_dev = devintr();
    // give up the CPU if a timer interrupt.
    if(which_dev == 2 && myproc() != 0)
        yield();
    // .....
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++){
            // .....
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

xv6 kernel is booting

Task 0: Created!
Task 0: Running...

```
void user_task1(void)
{
    printf("Task 1: Created!\n");
    while (1) {
        printf("Task 1: Running...\n");
        task_delay(DELAY);
    }
}
```

```
void kerneltrapret(void)
{
    struct proc *p = myproc();
    unsigned long x = r_sstatus();
    x |= SSTATUS_SPP;
    x |= SSTATUS_SPIE;
    w_sstatus(x);
    w_sepc((uint64)p->start);
    asm volatile("sret");
}
```

改进用户任务的启动方式 - 流程剖析

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
    }
}
```

```
kernelvec:
    # save context .....
    call kerneltrap
    # restore context .....
    sret
```

```
void kerneltrap()
{
    int which_dev = 0;
    // .....
    which_dev = devintr();
    // give up the CPU if a timer interrupt.
    if(which_dev == 2 && myproc() != 0)
        yield();
    // .....
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

xv6 kernel is booting

Task 0: Created!
Task 0: Running...
Task 1: Created!

```
void user_task1(void)
{
    printf("Task 1: Created!\n");
    while (1) {
        printf("Task 1: Running...\n");
        task_delay(DELAY);
    }
}
```

改进用户任务的启动方式 - 流程剖析

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
    }
}
```

```
kernelvec:
    # save context .....
    call kerneltrap
    # restore context .....
    sret
```

```
void kerneltrap()
{
    int which_dev = 0;
    // .....
    which_dev = devintr();
    // give up the CPU if a timer interrupt.
    if(which_dev == 2 && myproc() != 0)
        yield();
    // .....
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

```
void user_task1(void)
{
    printf("Task 1: Created!\n");
    while (1) {
        printf("Task 1: Running...\n");
        task_delay(DELAY);
    }
}
```

xv6 kernel is booting

Task 0: Created!
Task 0: Running...
Task 1: Created!
Task 1: Running...

A decorative graphic on the left side of the slide. It features two concentric circles with a light blue glow. A thin grey arc with dots at its ends curves around the top and right of the circles. In the top right corner, there is a solid blue rectangle.

04

抢占式多任务运行剖析

基于定时器中断的多任务抢占

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
    }
}
```

```
kernelvec:
    # save context .....
    call kerneltrap
    # restore context .....
    sret
```

```
void kerneltrap()
{
    int which_dev = 0;
    // .....
    which_dev = devintr();
    // give up the CPU if a timer interrupt.
    if(which_dev == 2 && myproc() != 0)
        yield();
    // .....
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

```
void user_task1(void)
{
    printf("Task 1: Created!\n");
    while (1) {
        printf("Task 1: Running...\n");
        task_delay(DELAY);
    }
}
```

xv6 kernel is booting

Task 0: Created!
Task 0: Running...
Task 1: Created!
Task 1: Running...

基于定时器中断的多任务抢占

```
void user_task0(void)
```

```
{  
    printf("Task 0: Created!\n");  
    while (1) {  
        printf("Task 0: Running...\n");  
        task_delay(DELAY);  
    }  
}
```

```
kernelvec:
```

```
# save context .....  
call kerneltrap  
# restore context .....  
sret
```

```
void kerneltrap()
```

```
{  
    int which_dev = 0;  
    // .....  
    which_dev = devintr();  
    // give up the CPU if a timer interrupt.  
    if(which_dev == 2 && myproc() != 0)  
        yield();  
    // .....  
}
```

```
void scheduler(void)
```

```
{  
    struct proc *p;  
    struct cpu *c = mycpu();  
    c->proc = 0;  
    for(;;){  
        for(p = proc; p < &proc[NPROC]; p++) {  
            // .....  
            c->proc = p;  
            switch(&c->context, &p->context);  
            c->proc = 0;  
            // .....  
        }  
    }  
}
```

xv6 kernel is booting

```
Task 0: Created!  
Task 0: Running...  
Task 1: Created!  
Task 1: Running...
```

```
void user_task1(void)
```

```
{  
    printf("Task 1: Created!\n");  
    while (1) {  
        printf("Task 1: Running...\n");  
        task_delay(DELAY);  
    }  
}
```

```
kernelvec:
```

```
# save context .....  
call kerneltrap  
# restore context .....  
sret
```

```
void kerneltrap()
```

```
{  
    int which_dev = 0;  
    // .....  
    which_dev = devintr();  
    // give up the CPU if a timer interrupt.  
    if(which_dev == 2 && myproc() != 0)  
        yield();  
    // .....  
}
```

基于定时器中断的多任务抢占

```
void user_task0(void)
```

```
{  
    printf("Task 0: Created!\n");  
    while (1) {  
        printf("Task 0: Running...\n");  
        task_delay(DELAY);  
    }  
}
```

```
kernelvec:
```

```
    # save context .....  
    call kerneltrap  
    # restore context .....  
    sret
```

```
void kerneltrap()
```

```
{  
    int which_dev = 0;  
    // .....  
    which_dev = devintr();  
    // give up the CPU if a timer interrupt.  
    if(which_dev == 2 && myproc() != 0)  
        yield();  
    // .....  
}
```

```
void scheduler(void)
```

```
{  
    struct proc *p;  
    struct cpu *c = mycpu();  
    c->proc = 0;  
    for(;;){  
        for(p = proc; p < &proc[NPROC]; p++) {  
            // .....  
            c->proc = p;  
            switch(&c->context, &p->context);  
            c->proc = 0;  
            // .....  
        }  
    }  
}
```

xv6 kernel is booting

```
Task 0: Created!  
Task 0: Running...  
Task 1: Created!  
Task 1: Running...
```

```
void user_task1(void)
```

```
{  
    printf("Task 1: Created!\n");  
    while (1) {  
        printf("Task 1: Running...\n");  
        task_delay(DELAY);  
    }  
}
```

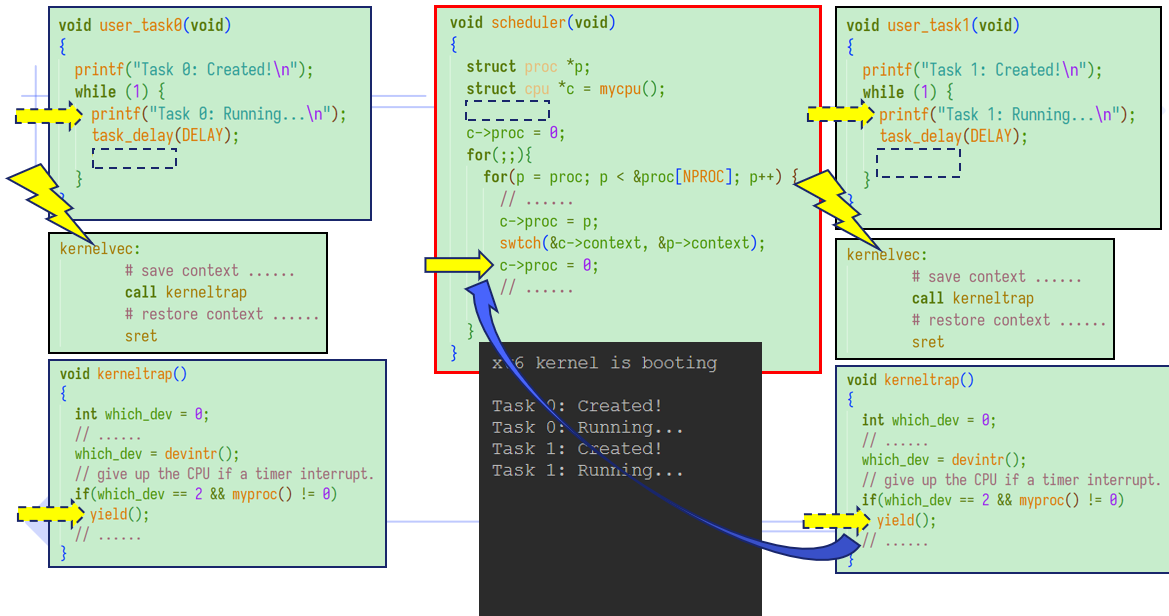
```
kernelvec:
```

```
    # save context .....  
    call kerneltrap  
    # restore context .....  
    sret
```

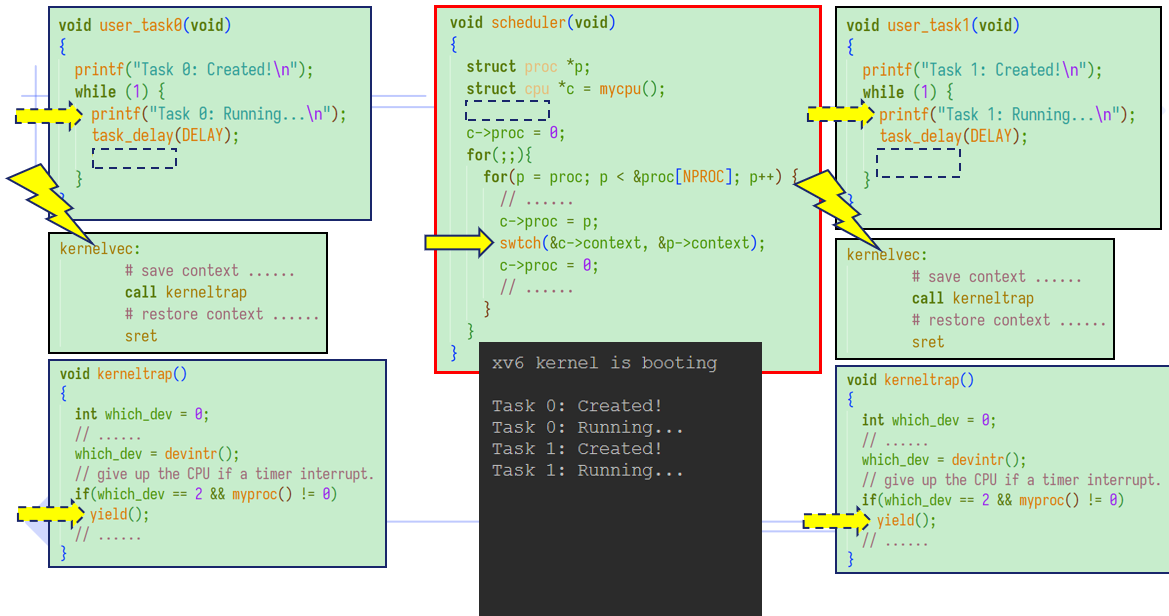
```
void kerneltrap()
```

```
{  
    int which_dev = 0;  
    // .....  
    which_dev = devintr();  
    // give up the CPU if a timer interrupt.  
    if(which_dev == 2 && myproc() != 0)  
        yield();  
    // .....  
}
```

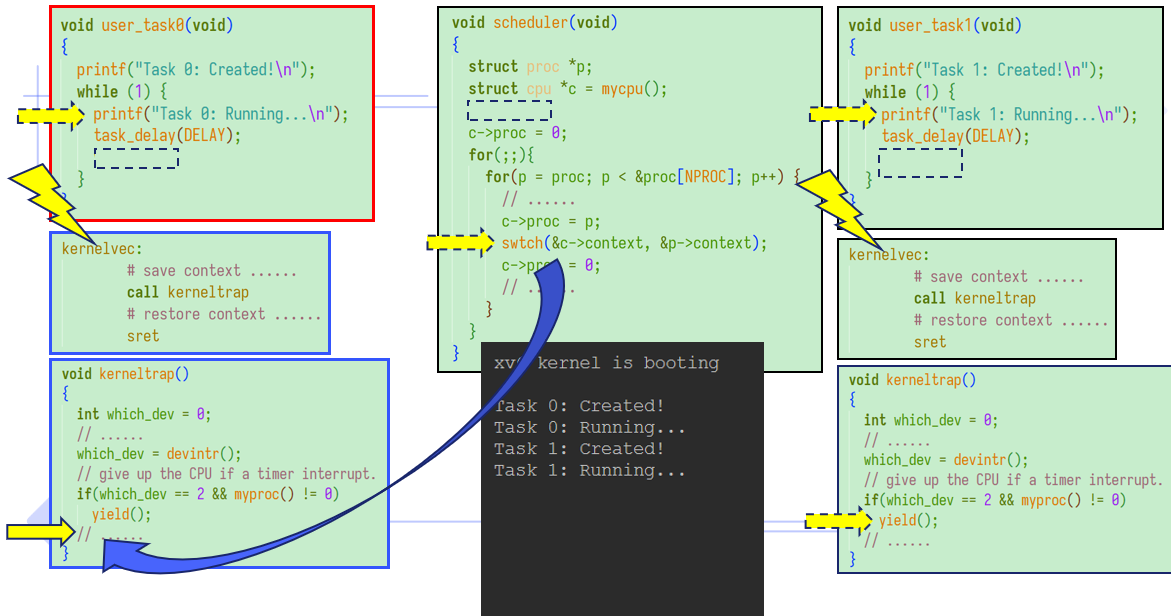
基于定时器中断的多任务抢占



基于定时器中断的多任务抢占



基于定时器中断的多任务抢占



基于定时器中断的多任务抢占

```
void user_task0(void)
```

```
{  
    printf("Task 0: Created!\n");  
    while (1) {  
        printf("Task 0: Running...\n");  
        task_delay(DELAY);  
    }  
}
```

```
kernelvec:
```

```
    # save context .....  
    call kerneltrap  
    # restore context .....  
    sret
```

```
void scheduler(void)
```

```
{  
    struct proc *p;  
    struct cpu *c = mycpu();  
    c->proc = 0;  
    for(;;){  
        for(p = proc; p < &proc[NPROC]; p++) {  
            // .....  
            c->proc = p;  
            switch(&c->context, &p->context);  
            c->proc = 0;  
            // .....  
        }  
    }  
}
```

```
xv6 kernel is booting
```

```
Task 0: Created!  
Task 0: Running...  
Task 1: Created!  
Task 1: Running...
```

```
void user_task1(void)
```

```
{  
    printf("Task 1: Created!\n");  
    while (1) {  
        printf("Task 1: Running...\n");  
        task_delay(DELAY);  
    }  
}
```

```
kernelvec:
```

```
    # save context .....  
    call kerneltrap  
    # restore context .....  
    sret
```

```
void kerneltrap()
```

```
{  
    int which_dev = 0;  
    // .....  
    which_dev = devintr();  
    // give up the CPU if a timer interrupt.  
    if(which_dev == 2 && myproc() != 0)  
        yield();  
    // .....  
}
```

基于定时器中断的多任务抢占

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
    }
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

```
void user_task1(void)
{
    printf("Task 1: Created!\n");
    while (1) {
        printf("Task 1: Running...\n");
        task_delay(DELAY);
    }
}
```

```
kernelvec:
    # save context .....
    call kerneltrap
    # restore context .....
    sret
```

xv6 kernel is booting

```
Task 0: Created!
Task 0: Running...
Task 1: Created!
Task 1: Running...
```

```
void kerneltrap()
{
    int which_dev = 0;
    // .....
    which_dev = devintr();
    // give up the CPU if a timer interrupt.
    if(which_dev == 2 && myproc() != 0)
        yield();
    // .....
}
```

基于定时器中断的多任务抢占

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
    }
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

```
void user_task1(void)
{
    printf("Task 1: Created!\n");
    while (1) {
        printf("Task 1: Running...\n");
        task_delay(DELAY);
    }
}
```

```
kernelvec:
    # save context .....
    call kerneltrap
    # restore context .....
    sret
```

xv6 kernel is booting

```
Task 0: Created!
Task 0: Running...
Task 1: Created!
Task 1: Running...
Task 0: Running...
```

```
void kerneltrap()
{
    int which_dev = 0;
    // .....
    which_dev = devintr();
    // give up the CPU if a timer interrupt.
    if(which_dev == 2 && myproc() != 0)
        yield();
    // .....
}
```

基于定时器中断的多任务抢占

```
void user_task0(void)
```

```
{  
    printf("Task 0: Created!\n");  
    while (1) {  
        printf("Task 0: Running...\n");  
        task_delay(DELAY);  
    }  
}
```

```
kernelvec:
```

```
    # save context .....  
    call kerneltrap  
    # restore context .....  
    sret
```

```
void kerneltrap()
```

```
{  
    int which_dev = 0;  
    // .....  
    which_dev = devintr();  
    // give up the CPU if a timer interrupt.  
    if(which_dev == 2 && myproc() != 0)  
        yield();  
    // .....  
}
```

```
void scheduler(void)
```

```
{  
    struct proc *p;  
    struct cpu *c = mycpu();  
    c->proc = 0;  
    for(;;){  
        for(p = proc; p < &proc[NPROC]; p++) {  
            // .....  
            c->proc = p;  
            switch(&c->context, &p->context);  
            c->proc = 0;  
            // .....  
        }  
    }  
}
```

xv6 kernel is booting

```
Task 0: Created!  
Task 0: Running...  
Task 1: Created!  
Task 1: Running...  
Task 0: Running...
```

```
void user_task1(void)
```

```
{  
    printf("Task 1: Created!\n");  
    while (1) {  
        printf("Task 1: Running...\n");  
        task_delay(DELAY);  
    }  
}
```

```
kernelvec:
```

```
    # save context .....  
    call kerneltrap  
    # restore context .....  
    sret
```

```
void kerneltrap()
```

```
{  
    int which_dev = 0;  
    // .....  
    which_dev = devintr();  
    // give up the CPU if a timer interrupt.  
    if(which_dev == 2 && myproc() != 0)  
        yield();  
    // .....  
}
```

基于定时器中断的多任务抢占

```
void user_task0(void)
```

```
{  
    printf("Task 0: Created!\n");  
    while (1) {  
        printf("Task 0: Running...\n");  
        task_delay(DELAY);  
    }  
}
```

```
kernelvec:
```

```
# save context .....  
call kerneltrap  
# restore context .....  
sret
```

```
void kerneltrap()
```

```
{  
    int which_dev = 0;  
    // .....  
    which_dev = devintr();  
    // give up the CPU if a timer interrupt.  
    if(which_dev == 2 && myproc() != 0)  
        yield();  
    // .....  
}
```

```
void scheduler(void)
```

```
{  
    struct proc *p;  
    struct cpu *c = mycpu();  
    c->proc = 0;  
    for(;;){  
        for(p = proc; p < &proc[NPROC]; p++) {  
            // .....  
            c->proc = p;  
            switch(&c->context, &p->context);  
            c->proc = 0;  
            // .....  
        }  
    }  
}
```

xv6 kernel is booting

```
Task 0: Created!  
Task 0: Running...  
Task 1: Created!  
Task 1: Running...  
Task 0: Running...
```

```
void user_task1(void)
```

```
{  
    printf("Task 1: Created!\n");  
    while (1) {  
        printf("Task 1: Running...\n");  
        task_delay(DELAY);  
    }  
}
```

```
kernelvec:
```

```
# save context .....  
call kerneltrap  
# restore context .....  
sret
```

```
void kerneltrap()
```

```
{  
    int which_dev = 0;  
    // .....  
    which_dev = devintr();  
    // give up the CPU if a timer interrupt.  
    if(which_dev == 2 && myproc() != 0)  
        yield();  
    // .....  
}
```

基于定时器中断的多任务抢占

```
void user_task0(void)
```

```
{  
    printf("Task 0: Created!\n");  
    while (1) {  
        printf("Task 0: Running...\n");  
        task_delay(DELAY);  
    }  
}
```

```
kernelvec:
```

```
    # save context .....  
    call kerneltrap  
    # restore context .....  
    sret
```

```
void kerneltrap()
```

```
{  
    int which_dev = 0;  
    // .....  
    which_dev = devintr();  
    // give up the CPU if a timer interrupt.  
    if(which_dev == 2 && myproc() != 0)  
        yield();  
    // .....  
}
```

```
void scheduler(void)
```

```
{  
    struct proc *p;  
    struct cpu *c = mycpu();  
    c->proc = 0;  
    for(;;){  
        for(p = proc; p < &proc[NPROC]; p++) {  
            // .....  
            c->proc = p;  
            swtch(&c->context, &p->context);  
            c->proc = 0;  
            // .....  
        }  
    }  
}
```

xv6 kernel is booting

```
Task 0: Created!  
Task 0: Running...  
Task 1: Created!  
Task 1: Running...  
Task 0: Running...
```

```
void user_task1(void)
```

```
{  
    printf("Task 1: Created!\n");  
    while (1) {  
        printf("Task 1: Running...\n");  
        task_delay(DELAY);  
    }  
}
```

```
kernelvec:
```

```
    # save context .....  
    call kerneltrap  
    # restore context .....  
    sret
```

```
void kerneltrap()
```

```
{  
    int which_dev = 0;  
    // .....  
    which_dev = devintr();  
    // give up the CPU if a timer interrupt.  
    if(which_dev == 2 && myproc() != 0)  
        yield();  
    // .....  
}
```


基于定时器中断的多任务抢占

```
void user_task0(void)
```

```
{  
    printf("Task 0: Created!\n");  
    while (1) {  
        printf("Task 0: Running...\n");  
        task_delay(DELAY);  
    }  
}
```

```
kernelvec:
```

```
# save context .....  
call kerneltrap  
# restore context .....  
sret
```

```
void kerneltrap()
```

```
{  
    int which_dev = 0;  
    // .....  
    which_dev = devintr();  
    // give up the CPU if a timer interrupt.  
    if(which_dev == 2 && myproc() != 0)  
        yield();  
    // .....  
}
```

```
void scheduler(void)
```

```
{  
    struct proc *p;  
    struct cpu *c = mycpu();  
    c->proc = 0;  
    for(;;){  
        for(p = proc; p < &proc[NPROC]; p++) {  
            // .....  
            c->proc = p;  
            switch(&c->context, &p->context);  
            c->proc = 0;  
            // .....  
        }  
    }  
}
```

xv6 kernel is booting

```
Task 0: Created!  
Task 0: Running...  
Task 1: Created!  
Task 1: Running...  
Task 0: Running...
```

```
void user_task1(void)
```

```
{  
    printf("Task 1: Created!\n");  
    while (1) {  
        printf("Task 1: Running...\n");  
        task_delay(DELAY);  
    }  
}
```

```
kernelvec:
```

```
# save context .....  
call kerneltrap  
# restore context .....  
sret
```

```
void kerneltrap()
```

```
{  
    int which_dev = 0;  
    // .....  
    which_dev = devintr();  
    // give up the CPU if a timer interrupt.  
    if(which_dev == 2 && myproc() != 0)  
        yield();  
    // .....  
}
```

基于定时器中断的多任务抢占

```
void user_task0(void)
```

```
{  
    printf("Task 0: Created!\n");  
    while (1) {  
        printf("Task 0: Running...\n");  
        task_delay(DELAY);  
    }  
}
```

```
kernelvec:
```

```
# save context .....  
call kerneltrap  
# restore context .....  
sret
```

```
void kerneltrap()
```

```
{  
    int which_dev = 0;  
    // .....  
    which_dev = devintr();  
    // give up the CPU if a timer interrupt.  
    if(which_dev == 2 && myproc() != 0)  
        yield();  
    // .....  
}
```

```
void scheduler(void)
```

```
{  
    struct proc *p;  
    struct cpu *c = mycpu();  
    c->proc = 0;  
    for(;;){  
        for(p = proc; p < &proc[NPROC]; p++) {  
            // .....  
            c->proc = p;  
            switch(&c->context, &p->context);  
            c->proc = 0;  
            // .....  
        }  
    }  
}
```

xv6 kernel is booting

```
Task 0: Created!  
Task 0: Running...  
Task 1: Created!  
Task 1: Running..  
Task 0: Running...
```

```
void user_task1(void)
```

```
{  
    printf("Task 1: Created!\n");  
    while (1) {  
        printf("Task 1: Running...\n");  
        task_delay(DELAY);  
    }  
}
```

```
kernelvec:
```

```
# save context .....  
call kerneltrap  
# restore context .....  
sret
```

```
void kerneltrap()
```

```
{  
    int which_dev = 0;  
    // .....  
    which_dev = devintr();  
    // give up the CPU if a timer interrupt.  
    if(which_dev == 2 && myproc() != 0)  
        yield();  
    // .....  
}
```

基于定时器中断的多任务抢占

```
void user_task0(void)
```

```
{  
    printf("Task 0: Created!\n");  
    while (1) {  
        printf("Task 0: Running...\n");  
        task_delay(DELAY);  
    }  
}
```

```
kernelvec:
```

```
    # save context .....  
    call kerneltrap  
    # restore context .....  
    sret
```

```
void kerneltrap()
```

```
{  
    int which_dev = 0;  
    // .....  
    which_dev = devintr();  
    // give up the CPU if a timer interrupt.  
    if(which_dev == 2 && myproc() != 0)  
        yield();  
    // .....  
}
```

```
void scheduler(void)
```

```
{  
    struct proc *p;  
    struct cpu *c = mycpu();  
    c->proc = 0;  
    for(;;){  
        for(p = proc; p < &proc[NPROC]; p++) {  
            // .....  
            c->proc = p;  
            switch(&c->context, &p->context);  
            c->proc = 0;  
            // .....  
        }  
    }  
}
```

xv6 kernel is booting

```
Task 0: Created!  
Task 0: Running...  
Task 1: Created!  
Task 1: Running...  
Task 0: Running...
```

```
void user_task1(void)
```

```
{  
    printf("Task 1: Created!\n");  
    while (1) {  
        printf("Task 1: Running...\n");  
        task_delay(DELAY);  
    }  
}
```

```
kernelvec:
```

```
    # save context .....  
    call kerneltrap  
    # restore context .....  
    sret
```

基于定时器中断的多任务抢占

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
    }
}
```

```
kernelvec:
    # save context .....
    call kerneltrap
    # restore context .....
    sret
```

```
void kerneltrap()
{
    int which_dev = 0;
    // .....
    which_dev = devintr();
    // give up the CPU if a timer interrupt.
    if(which_dev == 2 && myproc() != 0)
        yield();
    // .....
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

```
void user_task1(void)
{
    printf("Task 1: Created!\n");
    while (1) {
        printf("Task 1: Running...\n");
        task_delay(DELAY);
    }
}
```

xv6 kernel is booting

```
Task 0: Created!
Task 0: Running...
Task 1: Created!
Task 1: Running...
Task 0: Running...
```

基于定时器中断的多任务抢占

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
    }
}
```

```
kernelvec:
    # save context .....
    call kerneltrap
    # restore context .....
    sret
```

```
void kerneltrap()
{
    int which_dev = 0;
    // .....
    which_dev = devintr();
    // give up the CPU if a timer interrupt.
    if(which_dev == 2 && myproc() != 0)
        yield();
    // .....
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

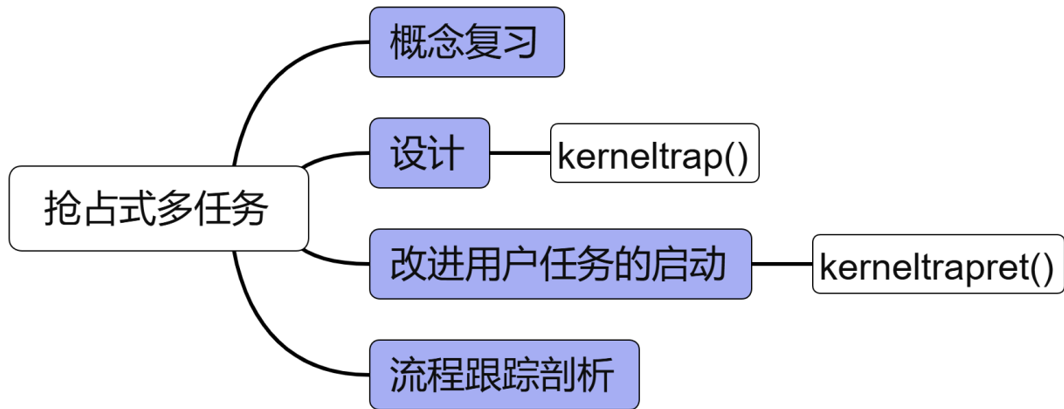
```
void user_task1(void)
{
    printf("Task 1: Created!\n");
    while (1) {
        printf("Task 1: Running...\n");
        task_delay(DELAY);
    }
}
```

xv6 kernel is booting

```
Task 0: Created!
Task 0: Running...
Task 1: Created!
Task 1: Running...
Task 0: Running...
Task 1: Running...
```



本章总结



谢谢

